

claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf\_b9d6fa25-e41\agent-a97cec7ff402c92d2.jsonl`
- SHA-256: `fac01a39ad8ddcd31956db153558454bfd56e7cdf539856477102b9109c1ee62`
- Source modified: `2026-06-25T17:39:21+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf\_b9d6fa25-e41`
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`

Transcript

1. user (2026-06-25T17:38:27.212Z)

Adversarially verify this PR-review finding for PR #385 (branch b1-p9/remove-fabricated-motion-data checked out; read the code ? STRICT READ-ONLY, do NOT edit any file). Be skeptical ? default to INVALID if wrong, already-handled, or a non-issue.

Finding [LOW] events table is now write-dead in production; event_type / server / receiver query filters silently return nothing for motion runs

Location: backend/infrastructure/database.py:612-633 (read) and backend/main.py:961-970 (/api/query)

Detail: Confirmed claim (b): after this PR the ONLY reference to add_event anywhere in backend/ is the method definition itself (database.py:498) ? motion was the sole production caller, so the events table is now never written by the pipeline (only tests write events). Reads degrade GRACEFULLY, no crash: query_play_segments builds r_dict["events"] = [] for motion segments (database.py:628-633), and the /api/query event_type filter (database.py:612-614, EXISTS subquery) simply matches zero segments. Likewise the server/receiver filters (database.py:604-610) and the /api/query server/receiver params (main.py:963-965) will never match motion segments now that those columns are None. This is a behavior change, not a break: the README still advertises 'Filter indexed play segments by server, receiver, ... and event_type' (README.md:26) but for the LIGHT-tier default segmenter those filters now always return empty. The honest-data tradeoff is correct (fabricated events were the bug), but the README/CODE_MAP claims are now stale for motion.

Suggestion: No code fix needed for safety. Optionally note in README.md:26 / docs/CODE_MAP.md:150 that server/receiver/event_type filters are populated only by AI/hybrid segmenters, not the motion LIGHT-tier default. The SPA is already null-safe (sample data ships server:null/receiver:null and shot_count null-guards), so no UI break.

Return isReal + adjustedSeverity (INVALID if not real) + one-paragraph reasoning grounded in the actual code.

2. assistant (2026-06-25T17:38:29.656Z)

I'll verify this finding by examining the actual code. Let me start by reading the relevant files.

3. user (2026-06-25T17:38:30.827Z)

b1-p9/remove-fabricated-motion-data
eabf774 fix(B1/P9): remove fabricated data from motion segmenter
cd5b3d2 feat(01/P11): Gemini remote-file GC ? startup orphan sweep (#380)
86705c1 chore: ruff format gemini_refine.py + trajectory_hybrid.py (#379)

4. user (2026-06-25T17:38:31.567Z)

```
490                                     (video_id, start_time, end_time, end_time - start_time, server,  
receiver, json.dumps(metadata or {}))
```

```

491         )
492         conn.commit()
493         return cursor.lastrowid
494     except Exception as e:
495         logger.error(f"Failed to add play segment: {e}")
496         return None
497
498     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
499         return self._execute_write(
500             "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
501             (segment_id, timestamp, event_type, player, json.dumps(details or {})),
502             "Failed to add event",
503         )
504
505     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
506                             chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
507                             stats: Optional[Dict[str, Any]] = None,
508                             detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
509         """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
510         detector-specific dicts stored as JSON. Returns the new candidate id, or
511         None."""
512         try:
513             with self._connect() as conn:
514                 cursor = conn.cursor()
515                 cursor.execute(
516                     """INSERT INTO candidate_segments
517                     (video_id, detector, chunk_index, start_time, end_time, duration,
518                     confidence, stats, detection_params)
519                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
520                     (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
521                     confidence, json.dumps(stats or {}), json.dumps(detection_params or
{})))
522                 conn.commit()
523                 return cursor.lastrowid
524     except Exception as e:
525         logger.error(f"Failed to add candidate segment: {e}")
526         return None
527
528     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
529         """Record that a candidate detection was confirmed as a given play segment."""
530         return self._execute_write(
531             "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id = ?",
532             (segment_id, candidate_id),
533             f"Failed to link candidate {candidate_id} to segment {segment_id}",
534         )
535
536     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
only_unlabeled: bool = False) -> List[Dict[str, Any]]:
537         """Fetch candidate detections for the annotation / fine-tuning workflow.
538         `stats` and `detection_params` are deserialized from JSON."""
539         query = "SELECT * FROM candidate_segments WHERE video_id = ?"
540         params: List[Any] = [video_id]
541
542         if detector:
543             query += " AND detector = ?"
544             params.append(detector)
545         if only_unlabeled:
546             query += " AND human_label IS NULL"

```

```

548
549         query += " ORDER BY start_time"
550
551         candidates = []
552         with self._connect() as conn:
553             rows = conn.cursor().execute(query, params).fetchall()
554             for row in rows:
555                 d = dict(row)
556                 d["stats"] = json.loads(d["stats"] or "{}")
557                 d["detection_params"] = json.loads(d["detection_params"] or "{}")
558                 candidates.append(d)
559         return candidates
560
561     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
562         with self._connect() as conn:
563             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
564 (video_id,)).fetchone()
565             if row:
566                 d = dict(row)
567                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
568                 return d
569             return None
570
571     def list_videos(self, limit: Optional[int] = None) -> List[Dict[str, Any]]:
572         """All indexed videos, most-recently-added first. Orders by rowid (a monotonic
573 insertion
574 counter, since id is a TEXT PK) rather than the 1-second-granularity
575 processed_at, which
576 would tie within a batch. Mirrors get_video's row shape (validation_results
577 parsed). Powers
578 GET /api/videos for the operator console (PACKAGING_PLAN.md P1)."""
579         sql = "SELECT * FROM videos ORDER BY rowid DESC"
580         params: tuple = ()
581         if limit is not None:
582             sql += " LIMIT ?"
583             params = (int(limit),)
584         with self._connect() as conn:
585             rows = conn.cursor().execute(sql, params).fetchall()
586             out: List[Dict[str, Any]] = []
587             for row in rows:
588                 d = dict(row)
589                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
590                 out.append(d)
591             return out
592
593     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
594 List[Dict[str, Any]]:
595         """
596         Dynamically queries play segments based on filters:
597         - duration_min: float
598         - server: string
599         - receiver: string
600         - event_type: string (e.g. smash, foul)
601         """
602         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
603         params = [video_id]
604
605         if filters.get("duration_min"):
606             query += " AND r.duration >= ?"
607             params.append(filters["duration_min"])
608
609         if filters.get("server"):
610             query += " AND r.server = ?"
611             params.append(filters["server"])
612

```

```

608         if filters.get("receiver"):
609             query += " AND r.receiver = ?"
610             params.append(filters["receiver"])
611
612         if filters.get("event_type"):
613             # Check if there is an event of a specific type inside this segment
614             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
615             params.append(filters["event_type"])
616
617         segments_list = []
618         with self._connect() as conn:
619             cursor = conn.cursor()
620             rows = cursor.execute(query, params).fetchall()
621
622             for row in rows:
623                 r_dict = dict(row)
624                 r_id = r_dict["id"]
625                 r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
626
627                 # Fetch associated events
628                 event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
629                 r_dict["events"] = []
630                 for er in event_rows:
631                     e_dict = dict(er)
632                     e_dict["details"] = json.loads(e_dict["details"] or "{}")
633                     r_dict["events"].append(e_dict)
634
635                 segments_list.append(r_dict)
636
637         return segments_list
638
639     def clear_video_data(self, video_id: str):
640         with self._connect() as conn:
641             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
642             conn.commit()
643
644     # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
645     # Writers follow the repo convention (swallow + logger.error, return bool); the
646     # worker logs loudly on a failed transition so a job can't silently wedge.
647
648     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
649                   player_pool: Optional[List[str]] = None,

```

5. assistant (2026-06-25T17:38:34.626Z)

The finding's claim (b) is that the only reference to `add\_event` in `backend/` is the method definition itself. Let me verify that, plus the diff of this PR and the README/CODE_MAP claims.

6. user (2026-06-25T17:38:35.690Z)

```

backend\infrastructure\database.py:498:     def add_event(self, segment_id: int, timestamp:
float, event_type: str, player: Optional[str] = None, details: Optional[Dict[str, Any]] = None)
-> bool:

```

7. user (2026-06-25T17:38:36.577Z)

```

Found 5 files
tests\test_motion_segmenter.py
tests\test_database.py
docs\CODE_MAP.md
backend\infrastructure\database.py
docs\ANALYZERS_AND_RECONCILERS.md

```

8. assistant (2026-06-25T17:38:39.411Z)

Confirmed: within `backend/`, the only `add\_event` reference is the method definition at database.py:498. Other matches are in tests and docs. Let me see the PR diff to confirm motion was the caller removed, and check main.py and the README/CODE_MAP claims.

9. user (2026-06-25T17:38:41.619Z)

```
backend/pipeline/segmenters/motion.py | 100 ++++++-----
tests/test_motion_segmenter.py         | 46 +++++-----
2 files changed, 39 insertions(+), 107 deletions(-)
=====DIFF=====
```

10. user (2026-06-25T17:38:42.049Z)

```
950         try:
951             path = resolve_under_root(path, output_dir)
952         except ValueError as e:
953             raise HTTPException(status_code=400, detail=str(e)) from e
954         if not os.path.isfile(path):
955             raise HTTPException(status_code=404,
956                               detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
957         media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
958         return FileResponse(path, media_type=media, filename=name)
959
960
961     @app.post("/api/query")
962     def query_play_segments(req: QueryRequest, request: Request):
963         filters = {
964             "server": req.server,
965             "receiver": req.receiver,
966             "duration_min": req.duration_min,
967             "event_type": req.event_type
968         }
969         segments = request.app.state.db.query_play_segments(req.video_id, filters)
970         return {"play_segments": segments}
971
972     class _CompileError(Exception):
973         """A submit-time-validatable compile problem ? carries the HTTP status + detail so
the API path
974         maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
result."""
975         def __init__(self, status_code: int, detail: str):
976             super().__init__(detail)
977             self.status_code = status_code
978             self.detail = detail
979
980
981     def _resolve_compile(db, cfg, video_id: str, segment_ids: List[int], output_filename:
str):
982         """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
resolve, since a
983         row could change between submit and run). Verifies the video exists, the requested
segments match
984         and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
shot_count are exempt
985         so manual/legacy picks can't silently vanish), and the output name is traversal-
safe. Raises
986         ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
safe_out_name)``."""
987         video = db.get_video(video_id)
988         if not video:
989             raise _CompileError(404, "Indexed video record not found.")
```

11. assistant (2026-06-25T17:38:45.513Z)

The diff shows the change is in `backend/pipeline/segmenters/motion.py` (not `motion\_segmenter.py`). Let me look at the actual motion.py diff to confirm `add\_event` calls were removed, and confirm server/receiver handling.

12. user (2026-06-25T17:38:46.493Z)

```
diff --git a/backend/pipeline/segmenters/motion.py b/backend/pipeline/segmenters/motion.py
index 9b714c5..36d6b36 100644
--- a/backend/pipeline/segmenters/motion.py
+++ b/backend/pipeline/segmenters/motion.py
@@ -1,5 +1,4 @@
 import logging
-import random
 import time
 from typing import Any, Dict, List, Optional, Tuple

@@ -22,12 +21,11 @@ class MotionPlaySegmenter(BasePlaySegmenter):

    def process_video(self, video_id: str, video_path: str, player_pool: Optional[List[str]] =
None, max_frames: Optional[int] = None) -> "Result[List[Dict[str, Any]]]":
        """
-        Processes a video to detect play segments using motion heuristics,
-        populates SQLite with detected segments, and indexes detailed events.
+        Processes a video to detect play segments using motion heuristics and
+        populates SQLite with detected segments. The ``player_pool`` arg is
+        accepted for ABC signature compatibility but no longer used (the motion
+        segmenter has no ground truth for server/receiver ? see _populate_db).
        """
-        if not player_pool:
-            player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
-
            cap = cv2.VideoCapture(video_path)
            if not cap.isOpened():
                logger.error(f"Segmenter could not open video: {video_path}")
@@ -35,7 +33,7 @@ class MotionPlaySegmenter(BasePlaySegmenter):

        final_segments = self._detect_motion_segments(cap, max_frames)

-        segments_data = self._populate_db_with_events(video_id, final_segments, player_pool)
+        segments_data = self._populate_db(video_id, final_segments)

        return Success(value=segments_data)

@@ -184,81 +182,35 @@ class MotionPlaySegmenter(BasePlaySegmenter):

        return final_segments

-    def _populate_db_with_events(self, video_id: str, final_segments: List[Tuple[float,
float]],
-                                player_pool: List[str]) -> List[Dict[str, Any]]:
-        """Persist ``final_segments`` and their fabricated metadata/events to the DB.
+    def _populate_db(self, video_id: str, final_segments: List[Tuple[float, float]]
+        ) -> List[Dict[str, Any]]:
+        """Persist ``final_segments`` to the DB with honest metadata.

-        For each detected segment this assigns a random server/receiver, fabricates
-        rich tags, then indexes a serve / 0?3 smash / termination event set ? all
-        with the stdlib ``random`` module (the legacy demo's fabricated baseline).
-        Returns the per-segment dicts handed back to the caller. The ``random``
-        call order here is load-bearing for reproducibility and must not change.
+        The motion segmenter detects play segments via pixel-diff heuristics but
+        has NO ground truth for who served, shot counts, shuttle speed, or event
+        breakdown ? those were previously fabricated with the stdlib ``random``

```

```

+ module (the legacy demo baseline). They are now persisted as ``None`` /
+ unknown markers so consumers can distinguish "not modeled" from a real
+ value. Matches yolo_hybrid/gemini, which pass ``server=None, receiver=None``.
+ """
-
- # Populate SQLite DB with segments and metadata events
- segments_data = []
- for start, end in final_segments:
-     # Assign Server and Receiver randomly from pool
-     server, receiver = random.sample(player_pool, 2)
-
-     # Formulate dynamic rich tags
-     dur = end - start
-     metadata = {
-         "shot_count": random.randint(4, 25),
+         "shot_count": None,
-         "intensity": "high" if dur > 12 else "medium",
-         "avg_speed_kmh": round(random.uniform(40, 85), 1)
+         "avg_speed_kmh": None,
-     }
-
-     segment_id = self.db.add_play_segment(video_id, start, end, server, receiver,
metadata)
-
-     if segment_id:
-         # Add sub-segment events
-         # A: Serve Event (always at start)
-         serve_foul = random.random() < 0.15 # 15% chance of serve foul
-         self.db.add_event(
-             segment_id=segment_id,
-             timestamp=start + 0.5,
-             event_type="serve",
-             player=server,
-             details={"status": "foul" if serve_foul else "valid", "type": "low_serve"
if random.random() > 0.5 else "high_serve"}
-         )
-
-         # B: Smash Events (1 to 3 random smashes per segment)
-         smashes_count = random.randint(0, 3)
-         for _ in range(smashes_count):
-             smash_time = random.uniform(start + 1.5, end - 1.0)
-             smasher = random.choice([server, receiver])
-             self.db.add_event(
-                 segment_id=segment_id,
-                 timestamp=round(smash_time, 2),
-                 event_type="smash",
-                 player=smasher,
-                 details={"speed_kmh": round(random.uniform(120, 250), 1)}
-             )
-
-         # C: Segment Termination Event (foul/winner)
-         foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
-         is_foul = random.random() > 0.4
-         error_player = random.choice([server, receiver])
-
-         self.db.add_event(
-             segment_id=segment_id,
-             timestamp=end - 0.2,
-             event_type="foul" if is_foul else "winner",
-             player=error_player,
-             details={"type": random.choice(foul_types)} if is_foul else {"shot_type":
"drop" if random.random() > 0.5 else "smash"}
-         )
-
-         # Store segment representation to return to caller
-         segments_data.append({

```

```

-         "id": segment_id,
-         "start_time": start,
-         "end_time": end,
-         "duration": dur,
-         "server": server,
-         "receiver": receiver,
-         "metadata": metadata
-     })
-
+     segment_id = self.db.add_play_segment(video_id, start, end, None, None, metadata)
+     segments_data.append({
+         "id": segment_id,
+         "start_time": start,
+         "end_time": end,
+         "duration": dur,
+         "server": None,
+         "receiver": None,
+         "metadata": metadata,
+     })
+     return segments_data

# Register the segmenter

```

13. assistant (2026-06-25T17:38:50.211Z)

The diff confirms the finding precisely: the motion segmenter previously called ``self.db.add_event`` (the only production caller) and now passes ``server=None, receiver=None`` with no events written. Let me verify the README and CODE_MAP claims, and confirm there are no other segmenters that call ``add_event``.

14. user (2026-06-25T17:38:51.573Z)

```

20     * `yolo_hybrid` ? two-stage pipeline. Stage 1 runs **torchvision person
detection** (Faster R-CNN, BSD-licensed) with **ByteTrack** tracking (supervision, MIT-licensed)
to cheaply filter the video down to a small set of high-motion candidate windows. Stage 2 hands
each candidate (with a small padding) to the AI provider for high-precision semantic boundaries
and shot counts. Massively cheaper than running the LLM on the full video. *(The registry key
remains `yolo_hybrid` for back-compat; the AGPL-licensed YOLOv8 was replaced with permissive
components ? see [docs/archives/MONETIZATION_AUDIT.md](docs/archives/MONETIZATION_AUDIT.md) and
ADR-005.)*
21     * `gemini` ? sends the entire video (or a `chunk_duration` slice) to Google Gemini
directly. Highest precision per call, highest cost.
22     * `motion` ? zero-dependency OpenCV pixel-difference heuristic. No API key
required. Useful as a baseline or for offline/air-gapped use.
23     * **Pluggable AI Provider Registry** (`backend/providers/`): The Gemini provider ships
in-tree; new providers (e.g. an OpenAI or local-model backend) can be registered without
touching the segmenters.
24     * **Pluggable Sport Profile Registry** (`backend/sports/`): The per-sport LLM prompt
and relevance config live in a `SportProfile`. Add a new sport by subclassing `SportProfile` and
registering it.
25     * **SQLite Indexing** : Each processed video is keyed by MD5 hash. Play segments
(start/end/duration/server/receiver/metadata) and per-segment events are persisted in
`output/sports_indexer.db`. Re-running on the same file is a cache hit.
26     * **Query System** : Filter indexed play segments by `server`, `receiver`,
`duration_min`, and `event_type` (the schema supports rich event types like
smashes/serves/fouls; today the shipped segmenters mainly populate timing + estimated
`shot_count`).
27     * **Zero-Dependency Local UI** : Served statically directly by FastAPI (Vite + Node.js
NOT required!) featuring a glassmorphic dashboard, responsive filtering, and immediate clip
stitching.
28     * **High-Speed FFmpeg Stitching** with configurable `begin_padding`, `end_padding`,
and `min_shot_count` filtering, plus an optional **branding watermark** burned into every reel
(**on by default** ? `Generated by Khelsutra.guru`, fully configurable, with an off switch).
29     * **Detailed CLI Debugging Tools** : `--debug-clip <t>` inspects frame metrics,
validator scores, and motion values around a timestamp. `--trim-start/--trim-end` extracts an

```


arbitrary segment via stream-copy ffmpeg for fast iteration.

30

31 ---

15. user (2026-06-25T17:38:52.510Z)

```
144 - `@backend/pipeline/segmenters/trajectory_hybrid.py::TrajectoryHybridSegmenter` ?
SHARED ENGINE for the trajectory hybrids (2026-06-11 collapse of the copy-paste twins; the "fix
BOTH" footgun is retired): healthcheck gate -> P2 trajectory->windows (+persist) -> P3 provider
handoff -> P4 DB/link; single `:::shot_count_from` (old wasb/tracknet shot_count drift gone).
NOT registered itself. ? Subclass contract: `family` (config section
`indexing.<family>.velocity_threshold(0.02)`, `output/<family>` trajectory dir,
`<family>-hybrid-<model>` metadata tag), `display_label` (log lines), `name`/`description`
properties, `_build_runner(idx_cfg) -> (DetectorRunner, detection_params extras)`. **Two fusion
seams (IDENTITY by default ? twins byte-identical, pinned by the equivalence test):
`:::enrich_candidate_stats(cw,points,fps,frame_width,video_path,idx_cfg)`** (base = the 4
motion-key stats dict; computed once into `window_stats`, used for persist + the handoff gate)
**and `:::select_for_handoff(windows>window_stats,idx_cfg)`** (base = all indices). ? shared
knobs still YOLO-named (`log_yolo_candidates/store_yolo_candidates`); whole-video single pass;
needs `{PROVIDER}_API_KEY` (missing -> []). Equivalence pinned by
`tests/test_trajectory_hybrid_equivalence.py` (detector identity, shot_count, offsets, + the
base-hook-identity tests ? parametrized over both twins).
145 - `@backend/pipeline/segmenters/fusion_hybrid.py::FusionSegmenter` (name
`fusion_hybrid`, **default-OFF** multi-signal fusion P2) ? subclasses the engine;
`family='fusion'`, `display_label='Fusion'`. `:::build_runner` selects the substrate via
`indexing.fusion.substrate` ('wasb'|'tracknet') reusing the existing runner+config.
`:::enrich_candidate_stats` overrides to ALWAYS add `net_crossings` (Gate A signal, GPU-free via
`rally_gate.gate_windows(..., estimate=self._axis_estimate(points))` ? axis/net cached per
trajectory) and, only when `fusion.cue_flags.person`, the ~5 person features
(`:::person_features` ? lazy `PersonDetector.detections_per_frame` over the window's ORIGINAL-
video frame range ? `fusion_features.compute_person_features`). `:::select_for_handoff`
overrides to apply served Gate A (`fusion.min_crossings`) ? Gate B (`fusion.min_players`,
person-only) ? default 0/0 ? all windows kept. ? persisted candidates are ALWAYS the full
superset (offline substrate intact); only the handoff/play_segments are gated. ? to A/B vs a
tuned wasb run, set `fusion.velocity_threshold = wasb.velocity_threshold` (the engine reads
`idx_cfg['fusion'].velocity_threshold`). ? person features eager even for gated windows (P3
cleanup). Tests: `tests/test_fusion_hybrid.py`.
146 - `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name
`wasb_hybrid`, the LIVE shuttle substrate; MIT, ships pretrained badminton weights) ? thin
subclass: `family='wasb'`, wires `WasbRunner`/`WasbConfig`; `detection_params` extras =
`weights_path` only.
147 - `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name
`tracknet_hybrid`) ? thin subclass: `family='tracknet'`, wires
`TrackNetRunner`/`TrackNetConfig`; extras = `weights_path` + `queue_length`.
148 - `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name
`yolo_hybrid`, no YOLO remains; ADR-005). **Stage-1 detection EXTRACTED to
`detectors/person_detector.py` (fusion P1, no behavior change)** ? the segmenter now
ORCHESTRATES only (chunking, pipelined-vs-sequential P2 prefetch where ffmpeg overlaps
sequential GPU, AI-provider handoff, candidate persist/link) and delegates detection to
`self.person` (`PersonDetector`). Imports `_DEFAULT_DETECTOR` from person_detector for the
`detection_params` snapshot. ? `yolo_tracker`/`bytetrack.yaml` is dead/back-compat; `gemini_`
config keys are deprecated aliases still honored; still mypy-quarantined for pre-existing
orchestration debt (the torchvision code that caused it left).
149 - `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `gemini`) ?
pure-AI, no P2. Chunks are RE-ENCODED + downscaled to `indexing.ai_chunk_scale_width` (default
1280; 0 = legacy stream-copy) before upload ? stream-copied 4K chunks exceed the provider's
`MAX_UPLOAD_MB` and every chunk gets refused ("0 segments / success"); re-encode also fixes
keyframe drift at chunk boundaries. Oversize refusals (provider metrics `oversize_skipped`) are
aggregated -> `logger.error` + `reporting.run_signals` (`oversize_clips_skipped`) for the report
rule. Workers from `indexing.ai_max_workers`. ? chunks only if `duration>chunk_duration`;
`max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts
colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning
table).
150 - `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `motion`) ?
pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM**
```

```
(random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing
server/receiver; uses `print()`; honors `max_frames`.
151
152     ### 2.2 Detectors & windowing (WSL-quarantined; ABC abstraction complete)
153     - `@backend/pipeline/detectors/base.py` ? DetectorRunner ABC (healthcheck() ->
Tuple[bool,str] never-raises; run_predict(...) -> Optional[str] CSV path). Contract deliberately
matches the pre-existing tuple returns and call sites in the hybrids. Rich module docstring
includes "how to add a new runner", Linux-native path notes (the de-risk goal), and guidance on
threading health into reports. Re-exported from `detectors/__init__.py`. This finishes the WSL
de-risk seam (low-regret 6 / review item 1).
154     - `@backend/pipeline/detectors/tracknet_runner.py`
155     - `::TrackNetConfig`(+`.from_indexing_cfg`; ? key drift `wsl_distro` JSON -> `distro`
field), `::TrackNetRunner` ? runs TrackNet `predict.py` in WSL.
```

16. user (2026-06-25T17:38:53.603Z)

No matches found

17. assistant (2026-06-25T17:39:04.297Z)

All the finding's claims are confirmed against the actual code:

1. ****Claim (b) ? only `add\_event` reference in `backend/` is the definition****: Confirmed. Grep over `backend/` returns only database.py:498 (the method def). The Grep over `backend/pipeline` returns no matches at all, and the PR diff shows the three `self.db.add\_event(...)` calls in `motion.py` were removed.
2. ****Motion was the sole production caller****: Confirmed. CODE_MAP.md:150 explicitly stated motion was the "only segmenter using `db.add\_event` and passing server/receiver". No other segmenter calls it.
3. ****Graceful read degradation****: Confirmed in code. `query\_play\_segments` builds `r\_dict["events"] = []` (database.py:629) and the `event\_type` EXISTS subquery (612-614) plus server/receiver equality filters (604-610) match zero motion segments now that those columns are `None`. No crash.
4. ****`/api/query` passes those params through****: Confirmed at main.py:963-967.
5. ****README/CODE_MAP staleness****: Confirmed. README.md:26 still advertises filtering by `server`, `receiver`, `event\_type`. CODE_MAP.md:150 still describes motion's events as "RANDOM ... only segmenter using `db.add\_event`" ? now stale since the PR removed exactly that behavior.

The finding is accurate, well-scoped, and correctly self-rates as LOW: it's a genuine behavior change (those filters silently return empty for the LIGHT-tier default segmenter) with stale docs, but reads degrade gracefully with no crash and no security/data-integrity impact. The README:26 caveat is partial ("today the shipped segmenters mainly populate timing + estimated `shot\_count`") but doesn't say the filters now return nothing ? and CODE_MAP:150 is now factually wrong about motion calling `add\_event`. This is a doc-staleness/observability nit rather than a functional bug; the honest-data tradeoff itself is correct.

18. user (2026-06-25T17:39:12.046Z)

Structured output provided successfully

19. assistant (2026-06-25T17:39:21.864Z)

Verified the finding against the actual code on the PR branch. It is ****real**** and correctly rated ****LOW****.

Key evidence:

- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py:498` ? the only `add\_event` reference remaining in `backend/` is the method definition; the PR diff removed all three production callers from `backend\pipeline\segmenters\motion.py`.
- `C:\Users\avidu\Projects\badminton-highlight-

indexer\backend\infrastructure\database.py:604-633` ? server/receiver/event\_type filters now match zero motion segments (columns are `None`, events list is `[]`); reads degrade gracefully, no crash.

- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py:961-970` ? `/api/query` passes those params straight through.
- Stale docs confirmed: `C:\Users\avidu\Projects\badminton-highlight-indexer\README.md:26` and `C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE\_MAP.md:150` (the latter still claims motion fabricates RANDOM events and is the only `add\_event` caller ? exactly what the PR deleted).

It is a doc-staleness/behavior-change nit, not a functional or security bug; the honest-data tradeoff itself is correct.